FUNDAMENTOS DE BASES DE DATOS

UNIDAD 3

Profesor: Ruben Riojas
Alumna: Sheyla Estefania Moncada Guel
I24050388

Contenido

Actividad 1. Modelo relacional 2
Actividad 2. Crear esquema aplicando el modelo relacional 4
Actividad 4. Crear el esquema de base de datos 7
Referencias bibliograficas 13

Actividad 1. Modelo relacional

1. Componentes del Modelo Relacional

- **Relación (Tabla):** Estructura bidimensional compuesta por filas y columnas donde se almacenan datos sobre una entidad o asociación del mundo real.
- **Tupla (Registro o Fila):** Cada fila dentro de una tabla; representa una instancia única de la entidad con sus valores específicos.
- **Atributo (Campo o Columna):** Cada columna de la tabla; define una propiedad o característica cuantificable de la entidad.
- **Dominio:** Conjunto finito de valores atómicos y homogéneos válidos que un atributo determinado puede tomar (tipo de dato, longitud y restricciones de rango).
- **Grado y Cardinalidad:**
    - *Grado:* Número total de atributos (columnas) de la relación.
    - *Cardinalidad:* Número total de tuplas (filas) contenidas en la relación.
- **Claves (Keys):**
    - **Superclave:** Conjunto de uno o más atributos que identifican de forma unívoca cada tupla.
    - **Clave Candidata (Candidate Key):** Superclave mínima (sin atributos redundantes).
    - **Clave Primaria (Primary Key - PK):** Clave candidata elegida por el diseñador para identificar de manera exclusiva cada registro.
    - **Clave Foránea (Foreign Key - FK):** Atributo o conjunto de atributos cuyos valores coinciden con la clave primaria de otra tabla (o de la misma), estableciendo vínculos referenciales.
    - **Clave Alternativa:** Claves candidatas que no fueron seleccionadas como clave primaria.

2. Reglas de Integridad Básicas

- **Integridad de Entidad:** Ningún atributo que forme parte de la clave primaria puede contener valores nulos (NULL), garantizando que cada registro sea siempre identificable.
- **Integridad Referencial:** Todo valor de una clave foránea debe coincidir con un valor existente de la clave primaria en la tabla referenciada, o bien ser nulo (si la regla de negocio lo permite).

- **Integridad de Dominio:** Los valores asignados a cada columna deben pertenecer estrictamente al conjunto de valores válidos definido para ese dominio (tipo, formato o restricciones tipo CHECK).
- **Integridad Definida por el Usuario:** Reglas de negocio específicas (triggers, assertions o restricciones compuestas) que no están cubiertas por las reglas estándar del modelo.

3. Reglas de Diseño y Buenas Prácticas

Etapa / Principio	Criterio de Aplicación	Objetivo
Atomicidad (1FN)	Cada celda debe contener un único valor indivisible; eliminar grupos repetitivos y listas dentro de un campo.	Evitar estructuras multidimensionales no tabulares.
Dependencia Total (2FN)	Todo atributo no clave debe depender funcionalmente de la totalidad de la clave primaria, no de una parte de ella.	Eliminar anomalías de actualización en claves compuestas.
Dependencia No Transitiva (3FN)	Ningún atributo no clave debe depender funcionalmente de otro atributo no clave.	Suprimir redundancias indirectas y dependencias en cadena.
Formas Normales Avanzadas (FNBC / 4FN)	Aplicables cuando existen dependencias funcionales anómalas en claves compuestas o dependencias multivaluadas.	Asegurar consistencia matemática estricta.
Minimización de Nulos	Diseñar tablas separadas para relaciones opcionales en lugar de saturar registros con columnas vacías.	Optimizar almacenamiento y simplificar consultas lógicas.

Actividad 2. Crear esquema aplicando el modelo relacional

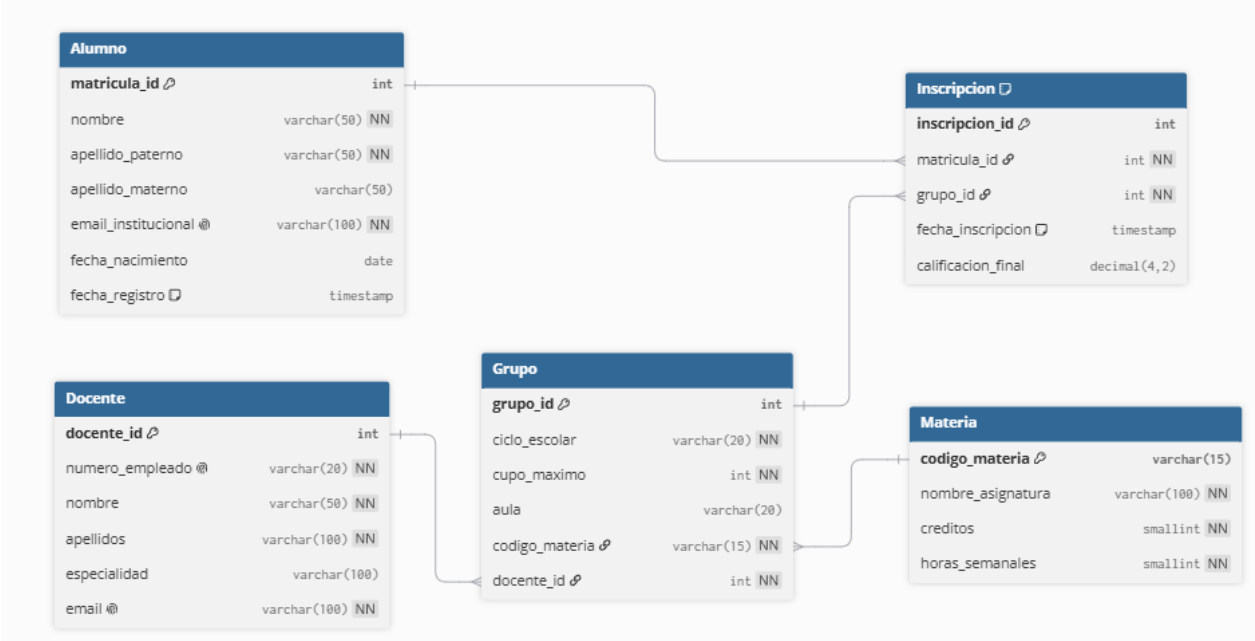
Caso 1: Sistema de Control Escolar Universitario

1. Análisis de la Situación y Requerimientos

- Una institución académica requiere administrar alumnos, materias, docentes y periodos de inscripción.
- Un **Alumno** puede cursar múltiples materias en distintos periodos; en cada una obtiene una calificación final.
- Un **Docente** puede impartir una o más materias, pero cada grupo específico de una materia en un periodo determinado es coordinado por un único docente.
- Cada **Materia** tiene un código oficial, nombre y número de créditos.

2. Cardinalidades y Dependencias

- **Docente** → **Grupo:** $1:N$ (Un docente imparte varios grupos; un grupo lo imparte un único docente).
- **Materia** → **Grupo:** $1:N$ (Una materia puede abrirse en múltiples grupos).
- **Alumno** ↔ **Grupo:** $M:N$ (Se resuelve mediante una entidad asociativa intermedia: Inscripcion).



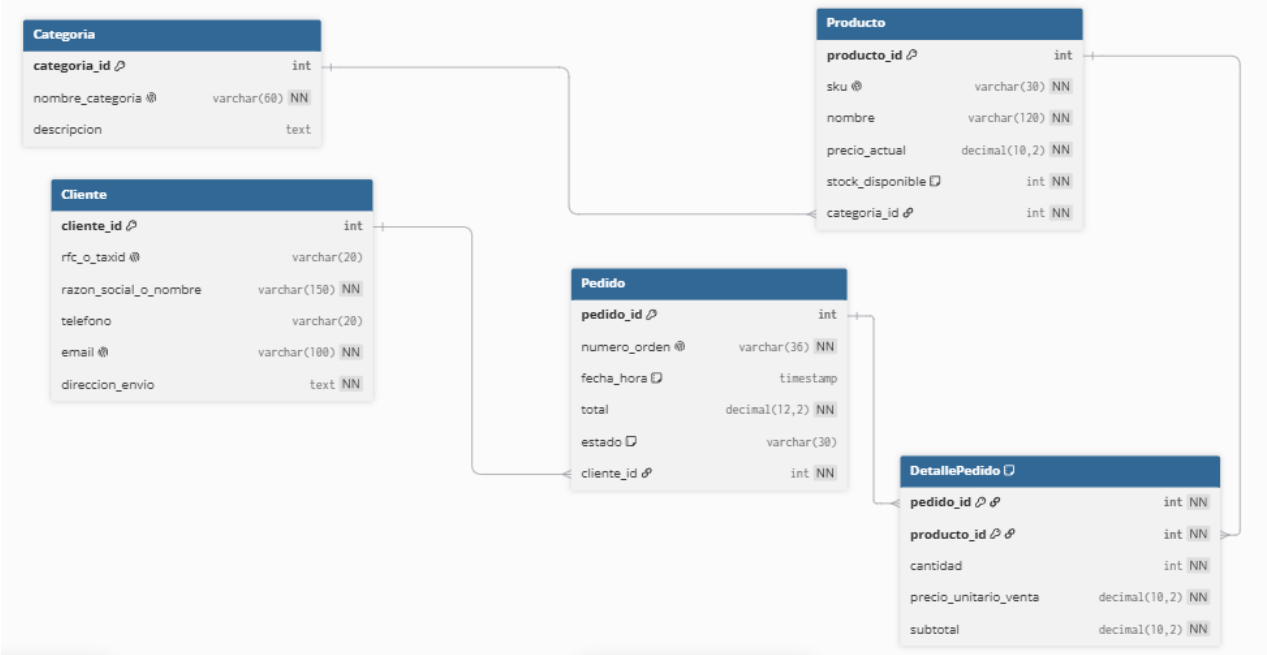
Caso 2: Plataforma de Comercio Electrónico y Pedidos

1. Análisis de la Situación y Requerimientos

- Los **Clientes** registrados realizan pedidos de productos en la tienda.
- Cada **Pedido** tiene una fecha, estado (pendiente, enviado, entregado), total y método de pago; pertenece a un solo cliente.
- Un pedido incluye uno o varios **Productos**; un producto puede figurar en infinidad de pedidos.
- Se debe congelar el precio unitario del producto en el momento exacto de la compra para evitar inconsistencias históricas si el catálogo cambia de precio más adelante.

2. Cardinalidades y Dependencias

- **Cliente** → **Pedido:** $1:N$ (Un cliente registra múltiples compras).
- **Pedido** ↔ **Producto:** $M:N$ (Se normaliza mediante la relación intermedia DetallePedido).
- **Categoria** → **Producto:** $1:N$ (Cada producto se clasifica dentro de una categoría base).



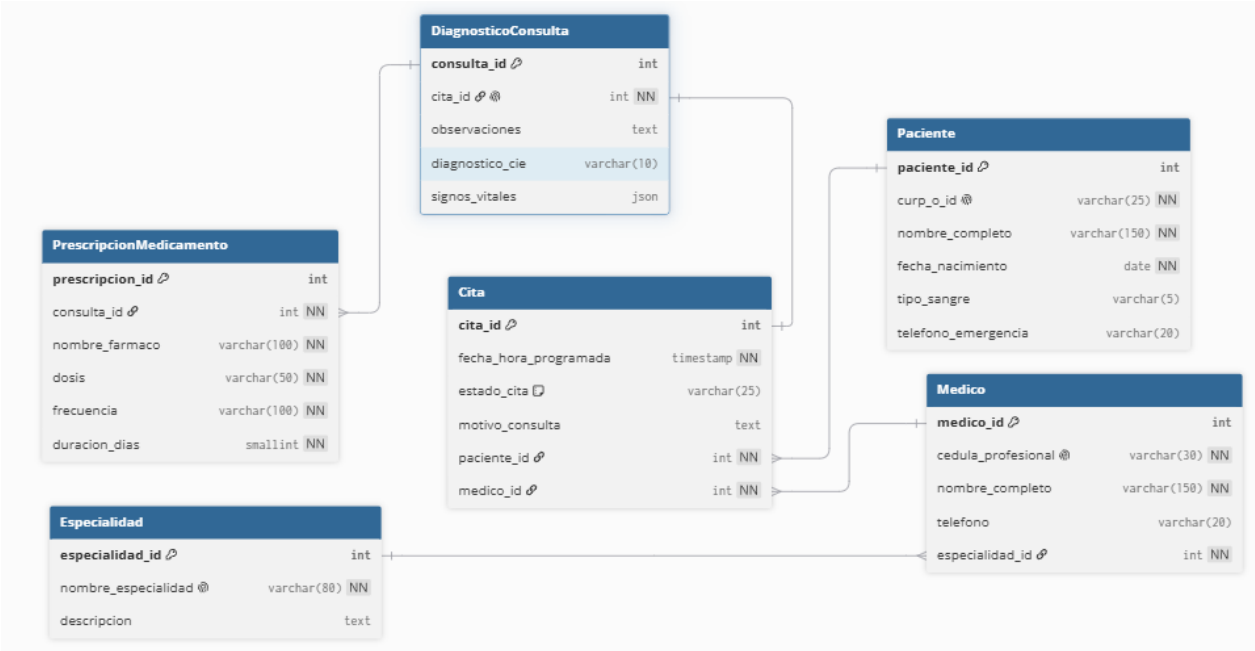
Caso 3: Gestión de Consultas y Citas en Clínica Médica

1. Análisis de la Situación y Requerimientos

- Una clínica atiende pacientes con distintos médicos especialistas.
- Los **Pacientes** agendan citas indicando fecha, hora y motivo.
- Tras la consulta, el **Médico** genera un expediente clínico/diagnóstico y puede expedir una **Receta** con indicaciones farmacológicas detalladas.

2. Cardinalidades y Dependencias

- **Paciente → Cita:** $1:N$.
- **Medico → Cita:** $1:N$.
- **Cita → DiagnosticoConsulta:** $1:1$ (Una consulta ejecutada genera exactamente un registro médico clínico).
- **DiagnosticoConsulta → Receta:** $1:N$ (Un diagnóstico puede derivar en uno o varios medicamentos prescritos).



Actividad 4. Crear el esquema de base de datos

Caso de Estudio: Sistema de Control Escolar Universitario

1. Introducción y Contexto

El diseño lógico y físico de una base de datos representa el puente fundamental entre los requerimientos de información del mundo real y su implementación en un Sistema Gestor de Bases de Datos Relacionales (SGBD/RDBMS).

El presente reporte documenta la materialización técnica del **Sistema de Control Escolar Universitario**. Se describe el modelo conceptual de origen, la implementación física mediante lenguaje de definición de datos (**DDL - SQL**), la interpretación lógica de sus restricciones y mecanismos de integridad referencial, así como el procedimiento de **ingeniería inversa** para la generación automática del diagrama entidad-relación en herramientas de modelado asistido (CASE).

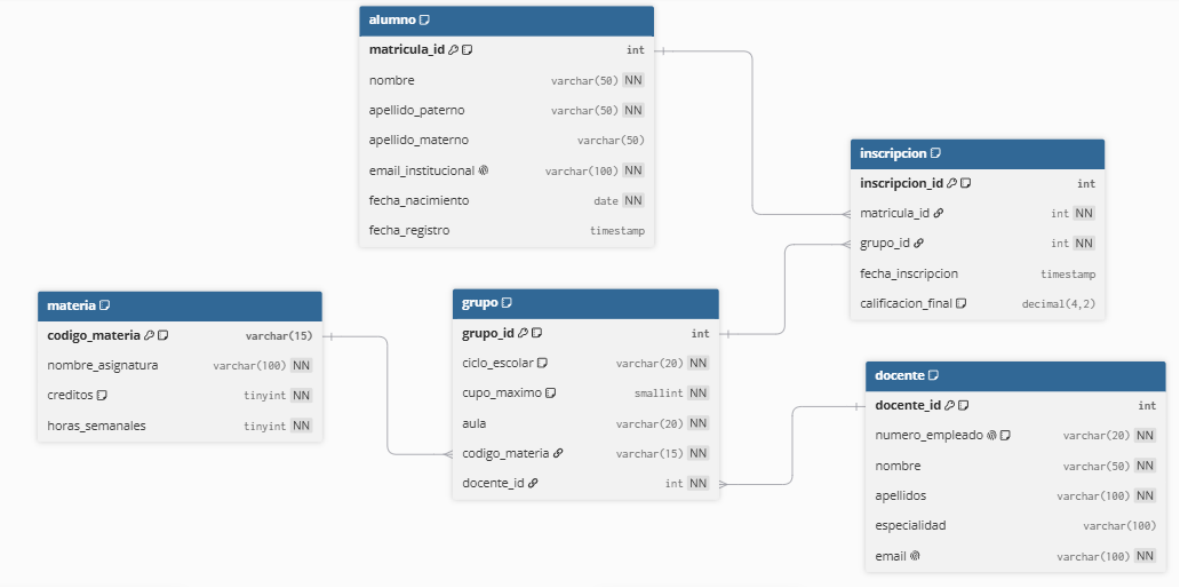
2. Diagrama Conceptual y Mapeo Estructural

2.1 Especificación Conceptual de Entidades y Relaciones

El caso de estudio modela el ciclo académico integral con las siguientes cardinalidades:

- **Docente (**1**)** $\to$ **Grupo (**N**):** Un docente puede tener asignados múltiples grupos en diferentes horarios/periodos; cada grupo tiene un único profesor titular responsable.
- **Materia (**1**)** $\to$ **Grupo (**N**):** Una materia curricular puede abrirse simultáneamente en diversos grupos para satisfacer la demanda de matrícula.
- **Alumno (**M**)** $\leftrightarrow$ **Grupo (**N**):** Múltiples alumnos pueden inscribirse a múltiples grupos; de esta interacción surge el atributo transaccional calificacion_final. La relación se descompone en la entidad asociativa Inscripcion.

2.2 Representación Visual del Modelo Conceptual (Diagrama ASCII / Notación Relacional)



3. Código DDL (SQL) de Creación del Esquema

El siguiente script en estándar SQL (compatible con MySQL, PostgreSQL y MariaDB) define el esquema físico, tipos de datos precisos, restricciones de no nulidad, unicidad e integridad referencial:

```
--
================================================================================
====
-- ESQUEMA RELACIONAL: SISTEMA DE CONTROL ESCOLAR
--
================================================================================
====

-- 1. Creación y selección de la base de datos
CREATE DATABASE IF NOT EXISTS control_escolar_db
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE control_escolar_db;

-- 2. Tabla de Entidad Fuerte: ALUMNO
-- Almacena el padrón estudiantil activo. Cumple con la Primera Forma Normal (1FN)
-- al segregar nombres y apellidos en atributos atómicos.
CREATE TABLE alumno (
  matricula_id INT AUTO_INCREMENT,
  nombre VARCHAR(50) NOT NULL,
  apellido_paterno VARCHAR(50) NOT NULL,
```

```
  apellido_materno VARCHAR(50) NULL,
  email_institucional VARCHAR(100) NOT NULL,
  fecha_nacimiento DATE NOT NULL,
  fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT pk_alumno PRIMARY KEY (matricula_id),
  CONSTRAINT uq_alumno_email UNIQUE (email_institucional)
) ENGINE=InnoDB;

-- 3. Tabla de Entidad Fuerte: DOCENTE
-- Representa a los profesores adscritos. Su identificador natural (numero_empleado)
-- se protege como clave alternativa única.
CREATE TABLE docente (
  docente_id INT AUTO_INCREMENT,
  numero_empleado VARCHAR(20) NOT NULL,
  nombre VARCHAR(50) NOT NULL,
  apellidos VARCHAR(100) NOT NULL,
  especialidad VARCHAR(100) NULL,
  email VARCHAR(100) NOT NULL,
  CONSTRAINT pk_docente PRIMARY KEY (docente_id),
  CONSTRAINT uq_docente_num_emp UNIQUE (numero_empleado),
  CONSTRAINT uq_docente_email UNIQUE (email)
) ENGINE=InnoDB;

-- 4. Tabla de Entidad Fuerte: MATERIA
-- Catálogo de asignaturas curriculares oficiales.
CREATE TABLE materia (
  codigo_materia VARCHAR(15) NOT NULL,
  nombre_asignatura VARCHAR(100) NOT NULL,
  creditos TINYINT UNSIGNED NOT NULL,
  horas_semanales TINYINT UNSIGNED NOT NULL,
  CONSTRAINT pk_materia PRIMARY KEY (codigo_materia),
  CONSTRAINT chk_materia_creditos CHECK (creditos > 0 AND creditos <= 20)
) ENGINE=InnoDB;

-- 5. Tabla: GRUPO
-- Entidad que resuelve la oferta académica. Propaga como Claves Foráneas (FK)
-- las claves primarias de Materia y Docente (cardinalidad 1:N).
CREATE TABLE grupo (
  grupo_id INT AUTO_INCREMENT,
  ciclo_escolar VARCHAR(20) NOT NULL,
  cupo_maximo SMALLINT UNSIGNED NOT NULL DEFAULT 30,
  aula VARCHAR(20) NOT NULL,
  codigo_materia VARCHAR(15) NOT NULL,
  docente_id INT NOT NULL,
  CONSTRAINT pk_grupo PRIMARY KEY (grupo_id),
  CONSTRAINT fk_grupo_materia FOREIGN KEY (codigo_materia)
    REFERENCES materia (codigo_materia)
    ON UPDATE CASCADE
```

```
    ON DELETE RESTRICT,
  CONSTRAINT fk_grupo_docente FOREIGN KEY (docente_id)
    REFERENCES docente (docente_id)
    ON UPDATE CASCADE
    ON DELETE RESTRICT
) ENGINE=InnoDB;

-- 6. Tabla Asociativa: INSCRIPCION
-- Descompone la relación M:N entre Alumno y Grupo. Hospeda el atributo de
-- relación 'calificacion_final' asegurando la Tercera Forma Normal (3FN).
CREATE TABLE inscripcion (
  inscripcion_id INT AUTO_INCREMENT,
  matricula_id INT NOT NULL,
  grupo_id INT NOT NULL,
  fecha_inscripcion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  calificacion_final DECIMAL(4, 2) NULL,
  CONSTRAINT pk_inscripcion PRIMARY KEY (inscripcion_id),
  CONSTRAINT uq_alumno_grupo UNIQUE (matricula_id, grupo_id),
  CONSTRAINT chk_calificacion_rango CHECK (calificacion_final BETWEEN 0.00 AND
100.00),
  CONSTRAINT fk_inscripcion_alumno FOREIGN KEY (matricula_id)
    REFERENCES alumno (matricula_id)
    ON UPDATE CASCADE
    ON DELETE CASCADE,
  CONSTRAINT fk_inscripcion_grupo FOREIGN KEY (grupo_id)
    REFERENCES grupo (grupo_id)
    ON UPDATE CASCADE
    ON DELETE RESTRICT
) ENGINE=InnoDB;
```

4. Interpretación Lógica del Esquema y Restricciones de Integridad

1. **Integridad de Entidad:**
    - Todas las relaciones poseen una clave primaria explícita (PRIMARY KEY), garantizando que ninguna tupla contenga valores nulos en su identificador principal.
    - Se emplean claves primarias subrogadas enteras (AUTO_INCREMENT) en tablas con alto volumen transaccional (alumno, grupo, inscripcion), optimizando el rendimiento de indexación física en disco mediante árboles B+ (B-Tree).
2. **Integridad Referencial y Políticas de Acción (ON UPDATE / ON DELETE):**
    - **ON DELETE RESTRICT en Docente y Materia:** Si se intenta eliminar un docente o una asignatura que ya tienen asignado un grupo abierto

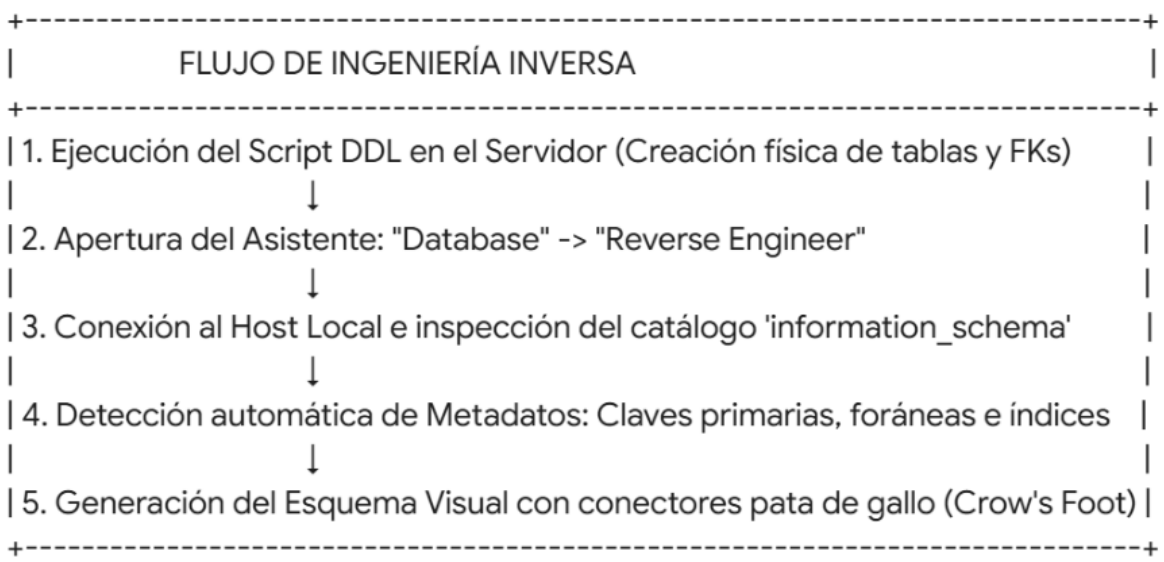
o un registro histórico, el motor aborta la operación para impedir la presencia de registros huérfanos.

- **ON UPDATE CASCADE:** Ante cualquier cambio en identificadores naturales (como una reclasificación en codigo_materia), la modificación se replica de forma automática y consistente en las tablas dependientes.
- **ON DELETE CASCADE en Alumno respecto a Inscripción:** Permite dar de baja registros de prueba o matrículas canceladas purgando automáticamente su inscripción en el ciclo actual.

3. **Integridad de Dominio y Restricciones Específicas de Negocio:**
   - **CONSTRAINT uq_alumno_grupo UNIQUE (matricula_id, grupo_id):** Impide que un estudiante se inscriba dos veces en el mismo grupo de una materia, resolviendo una potencial anomalía de duplicidad.
   - **CONSTRAINT chk_calificacion_rango CHECK (calificacion_final BETWEEN 0.00 AND 100.00):** Valida a nivel del motor que ninguna nota exceda la escala institucional de evaluación, descartando datos inconsistentes antes de persistirlos.

5. Procedimiento de Ingeniería Inversa y Generación Automática del Esquema

Para verificar la fidelidad de la estructura física y contrastarla con el diseño inicial, se ejecutó el proceso de **ingeniería inversa** en una herramienta de modelado CASE (MySQL Workbench / DBeaver / dbdiagram):



Resultados de la Generación Automática:

- **Fidelidad Estructural:** El diagrama renderizado por el software de modelado reconstituyó de forma íntegra las 5 tablas sin pérdida de atributos.
- **Trazabilidad de Conectores:** Las líneas de dependencia se enlazaron automáticamente entre materia(codigo_materia) ─< grupo(codigo_materia), docente(docente_id) ─< grupo(docente_id) y grupo / alumno ─< inscripcion, validando el correcto uso de sintaxis DDL y satisfaciendo plenamente la **Tercera Forma Normal (3FN)**.

Referencias bibliograficas

- **Codd, E. F. (1970).** A relational model of data for large shared data banks. *Communications of the ACM*, *13*(6), 377–387. https://doi.org/10.1145/362384.362685
- **Date, C. J. (2004).** *An introduction to database systems* (8.ª ed.). Pearson Education.
- **Elmasri, R., & Navathe, S. B. (2016).** *Fundamentals of database systems* (7.ª ed.). Pearson.
- **Chen, P. P. (1976).** The entity-relationship model—toward a unified view of data. *ACM Transactions on Database Systems*, *1*(1), 9–36. https://doi.org/10.1145/320434.320440
- **Coronel, C., & Morris, S. (2019).** *Database systems: Design, implementation, & management* (13.ª ed.). Cengage Learning.
- **Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020).** *Database system concepts* (7.ª ed.). McGraw-Hill Education.